P2408R4
Ranges iterators as inputs to non-Ranges algorithms

Published Proposal, 2021-11-16

This version:
https://wg21.link/p2408r4

Issue Tracking:
Inline In Spec

Author:
David Olsen (NVIDIA)

Audience:
SG9, LEWG

Toggle Diffs:
☐ Hide deleted text

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Table of Contents

§ 1. Abstract

Change the iterator requirements for non-Ranges algorithms. For forward iterators and above that are constant iterators, instead of requiring that iterators meet certain *Cpp17...Iterator* requirements, require that the iterators model certain iterator concepts. This makes iterators from several standard views usable with non-Ranges algorithms that require forward iterators or above, such as the parallel overloads of most algorithms.

2. Revision history

2.1. R0

This paper arose out of a private e-mail discussion where Bryce Adelstein Lelbach questioned why code that is similar to the first example in § 3 Motivation didn't compile with MSVC.

2.2. R1

Add the section § 8 Sentinels based on discussion in SG9 (Ranges) on 9 Aug 2021. This is just background information; there is no change to what is being proposed.

2.3. R2

Change the title from "Ranges views as inputs to non-Ranges algorithms" to "Ranges iterators as inputs to non-Ranges algorithms".

Based on discussion in SG9 (Ranges) on 13 Sep 2021, withdraw the portion of the paper that changed input iterators and output iterators to use iterator concepts. The requirements for input and output iterators remain unchanged from the current standard, using *Cpp17Iterator* requirements.

Add the section Mutable Iterators. This is just background information; there is no change to what is being proposed.

In § 4 Analysis, discuss the iterators for `zip_view`, et al., that are being proposed in [P2321].

2.4. R3

Based on discussion in SG9 (Ranges) on 11 Oct 2021, change the proposal to handle proxy iterators correctly. This turned out to be a significant change, both to the wording and to the design section. In the wording, the change from *Cpp17...Iterator* requirements to iterator concepts now only happens for constant iterators, not all iterators. In the design section, § 5.3 Algorithms require concepts instead of categories and § 6 Impact and Details had significant changes and § 7 Mutable Iterators and Proxy Iterators was completely rewritten.

§ 2.5. R4

Gather implementation experience, described in § 9 Implementation Experience. Update the § 6.4 Implementation impact section based on the implementation experience.

§ 3. Motivation

These two snippets of code should be well-formed:

```
std::vector<int> data = ...;
auto v = data | std::views::transform([](int x){ return x * x; });
int sum_of_squares = std::reduce(std::execution::par, begin(v), end(v));
```

```
auto idxs = std::views::iota(0, N);
std::transform(std::execution::par, begin(idxs), end(idxs), begin(sqrts),
               [](int x) { return std::sqrt(float(x)); });
```

It should be possible, in most cases, to use the iterators from ranges and views as the inputs to C++17 parallel algorithms and to other algorithms that require forward iterators or greater.

§ 4. Analysis

`std::reduce` requires that its iterator parameters satisfy the requirements of *Cpp17ForwardIterator*. One of those requirements is that `std::iterator_traits<I>::reference` be a reference type, either `T&` or `const T&`. ([forward.iterators]/p1.3) The iterator category of `transform_view::iterator` matches the iterator category of the underlying range's iterator only if the transformation function returns an lvalue reference. If the return type of the transformation function is a value rather than a reference, the iterator category of `transform_view::iterator` is `input_iterator_tag`, because the iterator's `reference` type alias can't be a reference type. ([range.transform.iterator]/p2)

In this example, the lambda transformation function returns a value, so the iterators passed to `std::reduce` are only input iterators in the classic iterator taxonomy used by the non-Ranges algorithms, even though the iterators satisfy the `random_access_iterator` concept in the Ranges iterator taxonomy.

Several other views have the same issue, where the iterator category of the view's iterator is `input_iterator_tag` even when the iterator concept is something else.

- `iota_view::iterator`'s iterator category is always `input_iterator_tag` while its iterator concept is often `random_access_iterator_tag`. [range.iota.iterator]
- `lazy_split_view::iterator`'s iterator category is always `input_iterator_tag` while its iterator concept is `forward_iterator_tag` if the underlying range is a forward range. [range.lazy.split.outer]
- `split_view::iterator`'s iterator category is always `input_iterator_tag` and its iterator concept is always `forward_iterator_tag`. [range.split.iterator]
- `elements_view::iterator` is similar to `transform_view::iterator` in that the iterator category depends on the value category of the return type of the `std::get<N>` function used to extract the element. The iterator category is `input_iterator_tag` if `std::get<N>` returns an rvalue, and (mostly) matches the iterator category of the base range's iterator otherwise. The iterator concept depends only on the base range's iterator, not on the `std::get<N>` function. [range.elements.iterator]/p2
- In [P2321], the iterators for `zip_view` and `adjacent_view` have an iterator category of `input_iterator_tag` but an iterator concept that matches the lowest iterator concept of the iterators of the underlying ranges. The iterators for `zip_transform_view` and `adjacent_transform_view` determine their category and concept similar to how it is done for `transform_view::iterator`.

§ 4.1. Current situation

MSVC checks that iterator arguments to parallel algorithms have the correct iterator category. If either of the code snippets in § 3 Motivation are compiled, the compilation will fail with "error C2338: Parallel algorithms require forward iterators or stronger."

libstd++ does not do any up-front checking of the iterator category of algorithm arguments. Its implementation of algorithms will accept iterators of any category as long as the iterators support the operations that are actually used in the implementation. The code snippets in § 3 Motivation can be successfully compiled with GCC 11.1.

5. Possible Solutions

5.1. Do nothing

We could simply wait for parallel versions of Range-based algorithms to make their way into the standard. Until then, users who want to use certain view iterators as inputs to parallel algorithms are out of luck.

This solution might be worth considering if parallel Range-based algorithms were on track for C++23. But [P2214] "A Plan for C++23 Ranges" puts parallel Range-based algorithms in Tier 2 of Ranges work, with the caveat that the work needs to be coordinated with Executors to make sure the interface is correct. That pushes the work well past C++23.

Even if parallel Range-based algorithms were already in the draft IS, it would still be worth investigating how to get Ranges iterators to work better with non-Ranges algorithms, in the interest of having the various parts of the Standard work well together.

5.2. Relax the *Cpp17* iterator requirements

We could change the definition of *Cpp17ForwardIterator* ([forward.iterators]), removing the requirement that `reference` be defined for constant iterators. `reference` would still need to be defined, and be `T&`, for mutable iterators.

The authors do not consider this option to be feasible. This change would break existing code that assumes that `iterator_traits<I>::reference` exists and is a reference type if the iterator category is `forward_iterator_tag` or greater. There are other possible solutions that don't break existing code.

5.3. Algorithms require concepts instead of categories

We could change the wording in [algorithms.requirements] to say that the template arguments of the algorithms shall model an iterator concept rather than meet a set of *Cpp17* requirements when the iterator is used as a constant iterator. For example:

> If an algorithm's template parameter is named `ForwardIterator`, `ForwardIterator1`, or `ForwardIterator2`, the template argument shall meet the *Cpp17ForwardIterator* requirements ([forward.iterators]) if it is required to be a mutable iterator, or model `forward_iterator` ([iterator.concept.forward]) otherwise .

This change relaxes the requirements on some of the template arguments for non-Ranges algorithms. Implementations may need to change to no longer depend on requirements that are no longer required. It is not expected that this will be a big burden, since a well-written algorithm shouldn't be depending on `iterator_traits<I>::reference` being a reference type for an iterator that is only used for input.

This change will not break any existing code because the iterator concept imposes fewer requirements on the type than the corresponding *Cpp17...Iterator* requirements. Any well-formed program will continue to be well-formed and have the same behavior. Some programs that were not well-formed may become well-formed, such as the two motivating examples in this paper; these will have reasonable, non-surprising behavior.

This is the solution proposed by this paper. See immediately below for more details and analysis of this solution.

Making this change only for constant iterators, not mutable iterators, is important. See § 7 Mutable Iterators and Proxy Iterators for details.

For reasons explained below, this change is proposed only for forward, bidirectional, and random access iterators. The requirements for input and output iterators remain unchanged.

§ 6. Impact and Details

§ 6.1. Changes

In the bullets of [algorithms.requirements]/p4 for forward, bidirectional, and random access iterators leave the requirements unchanged for mutable iterators and change the requirements on constant iterators from meeting a set of *Cpp17...Iterator* requirements to modeling the corresponding iterator concept.

§ 6.2. Relaxed requirements

The *Cpp17...Iterator* requirements and the iterator concepts are worded very differently, making it challenging to compare them directly.

The only thing that I can find that the iterator concepts require that the *Cpp17...Iterator* requirements do not is that iterator destructors must be declared `noexcept`. (*Cpp17Iterator* requires *Cpp17Destructible*, and *Cpp17Destructible* requires that the destructor not throw any exceptions. But *Cpp17Destructible* allows the destructor to be declared `noexcept(false)`, while the

`input_or_output_iterator` concept requires that the destructor be declared `noexcept(true)`.) Since destructors are implicitly declared `noexcept`, and no standard iterator has a `noexcept(false)` destructor, this difference is not meaningful and is not expected to cause any problems.

The one thing that *Cpp17ForwardIterator* requires that the `forward_iterator` concept does not is that `iterator_traits<I>::reference` be a reference type, either `T&` or `const T&`. `forward_iterator` requires that `iter_reference_t<I>` exist, but doesn't require that it be a reference type. This same difference also applies to bidirectional iterators and random access iterators. This difference is the root cause of the problems, and is the reason for this paper.

§ 6.3. Other uses

The *Cpp17...Iterator* requirements are used in several other places in the standard besides [algorithms.requirements]. It is proposed that in places where the requirements are on forward or above non-mutable iterator types that the program passes to standard algorithms, the wording is changed from meeting *Cpp17...Iterator* requirements to modeling an iterator concept, just like is being done for [algorithms.requirements]. See § 11 Wording for the five places where this happens. Sections where the only requirements on program-supplied iterators are *Cpp17InputIterator* or *Cpp17OutputIterator* are not changed. These are [rand.dist.samp.discrete], [rand.dist.samp.pconst], [rand.dist.samp.plinear], and [re.results.form]. Sections where the requirements on forward or above iterators are unchanged because they are required to be mutable are [rand.util.seedseq], [alg.shift], [alg.partitions].

Other uses of *Cpp17...Iterator* requirements are not changed by this proposal. In some of those cases the requirement is on a standard iterator, not a program iterator, so relaxing the requirements could break existing code. The other uses are in [sequence.reqmts], [associative.reqmts.general], [move.iter.requirements], [locale.category], [fs.req], [fs.path.req], [fs.class.directory.iterator.general], [time.zone.db.list], [reverse.iter.requirements], [allocator.requirements.general], [stacktrace.basic.obs], [string.view.iterators], [container.requirements.general], [span.iterators], [iterator.operations], [alg.equal], and [valarray.range].

§ 6.4. Implementation impact

If any standard algorithm implementations rely on the return type of dereferencing a forward (or higher), non-mutable iterator being a reference type (see § 6.2 Relaxed requirements) or the iterator's `value_type` (see § 7 Mutable Iterators and Proxy Iterators), those implementations will have to

change to not rely on that. Based on testing (see § 9 Implementation Experience), the number of changes of this sort will be small.

Any implementation, such as MSVC, that checks the iterator category of forward iterators passed to algorithms will have to change the way those checks happen. That change, while a little bit tedious, is not at all hard.

If an algorithm checks the iterator category to choose the most efficient implementation, in some cases that code should be changed to check the iterator concept instead. Testing turned up two cases of this in libstdc++ (see § 9 Implementation Experience), but testing probably won't find all cases like this. Visual code inspection might be necessary.

I expect that the biggest impact on implementations will be expanding their test suites to cover the code patterns that will become well-formed once this proposal is adopted.

§ 6.5. User impact

No well-formed programs will become ill-formed (unless there are user-defined iterators with `noexcept(false)` destructors) or have different behavior as a result of this change. Some ill-formed programs will become well-formed with the behaviors users would expect. This change will reduce the number of surprises by making code that users reasonably expect to be well-formed actually well-formed. It is not expected that any users would be negatively impacted by this change.

§ 7. Mutable Iterators and Proxy Iterators

The change from *Cpp17...Iterator* requirements to iterator concepts is only being done for constant iterators. If the requirements for mutable iterators were changed in the same way, that would allow the passing of proxy iterators to algorithms that wouldn't handle them correctly.

C++ has always had at least one standard proxy iterator: `vector<bool>::iterator`. C++23 will get several additional proxy iterators when [P2321] adds `zip` and related views to the standard library. Proxy iterators often satisfy the requirements of mutable iterators, because they have been designed so that `*it = v;` does what users expect. But things may go wrong, either compilation errors or unexpected runtime behavior, when other mutating operations are done on proxy iterators, such as `v = std::move(*it);` or `swap(*it1, *it2);`. For mutable non-proxy iterators, `decltype(*it)` is usually `T&`, but for mutable proxy iterators, `decltype(*it)` is something different. (For `zip_view::iterator`, I believe `decltype(*it)` is a `pair` or `tuple` of reference types, not a reference to a `pair` or `tuple`.)

Ranges algorithms are designed to correctly handle proxy iterators. For example, their specifications say that they call `iter_move` or `iter_swap`, which proxy iterator types can customize, when elements need to be moved or swapped. Non-Ranges algorithms are usually not prepared to handle proxy iterators, with their specifications calling `std::move` or `swap` directly on the dereferenced iterators. See the algorithms `move` and `swap_ranges` for examples of how `std::` algorithms and `std::ranges::` algorithms are specified differently.

Proxy iterators can be used as constant iterators without problems. It is only their use as mutable iterators in algorithms that aren't designed for them that causes problems.

Keeping the *Cpp17...Iterator* requirements for mutable iterators passed to non-Ranges algorithms prevents using proxy iterators in those situations from being well-formed, because proxy iterators do not satisfy the *Cpp17ForwardIterator* requirement that `iterator_traits<I>::reference` be `T&`.

§ 8. Sentinels

Many ranges use iterator/sentinel pairs, rather than a pair of iterators, as the bounds of the range. Non-Ranges algorithms always require a pair of iterators that have the same type, and will not work with iterator/sentinel pairs. If this proposal is adopted, more ranges than before will be usable with classic algorithms, but the sentinel issue will still prevent some ranges from being useful as inputs to classic algorithms.

The two examples in § 3 Motivation use iterator pairs and don't suffer from the sentinel issue. `transform_view::end()` returns an iterator rather than a sentinel if the base range is a common range. In the example, the base range is `std::vector`, which is a common range. `iota_view::end()` returns an iterator rather than a sentinel when the range has an upper bound of the same type as the lower bound. It is primarily unbounded `iota_view`s that use a sentinel. Passing an unbounded `iota_view` to any classic algorithm function as a first/last pair isn't going to work well, even if first and last are of the same type. I don't think `iota_view`'s use of sentinels will be a problem in practice in this area.

If a range with iterator/sentinel pairs needs to be used with a classic algorithm, there is a good chance that the user can simply wrap the range in a `common_view`, or wrap the range's iterator and sentinel in a `common_iterator`. The `common_view`'s iterators will be random access if the base range is random access and sized, and will be forward iterators if the base range's iterator is at least forward. That will allow the `common_view` to be used in most places where the base range would have been usable if it had used an iterator rather than a sentinel.

I am in favor of changing the non-Ranges algorithms to use iterator/sentinel pairs. But that change can be done independently, not as part of this paper. Both changes are useful on their own, independent of

the other, and wouldn't benefit from being tied together.

§ 9. Implementation Experience

I took a smallish test suite of the C++ parallel algorithms that I wrote a few years ago and adapted it to test this paper. I made two copies of the test suite. In one copy, I used `transform_view`'s iterators, or occasionally `iota_view`'s iterators, wherever possible. In the other copy, I used a hand-written proxy iterator adapter wherever possible. The proxy iterator was designed to have minimal functionality and to catch cases where code assumes that dereferencing an iterator gives you a reference type or a prvalue of the iterator's `value_type`.

I ran this test suite with GCC 11 in C++20 mode using its libstdc++. That turned up three places that will need to be changed if this proposal is adopted. In two algorithms, `find_end` and `search_n`, the code checked the iterator category and failed to compile if the category was not at least `forward_iterator_tag`. With this proposal, the code will have to be changed to check the iterator concept instead of iterator category. The other failure was in the overload of `inclusive_scan` with no initial value. That failed when passed a proxy iterator because it got the type of the intermediate value wrong.

I also ran the test suite with Visual Studio 2022 with /std:c++latest, with the standard library cloned from the STL project on GitHub. I edited the parallel algorithms in the library (see § 4.1 Current situation), changing the requirements checks on the iterators to match this proposal, which meant changing from checking that the iterator's category was at least `forward_iterator_tag` to checking that the iterator satisfied the `forward_iterator` concept. Once I did that, all the tests passed.

The parallel algorithms that were covered by my test suite are `adjacent_difference`, `adjacent_find`, `all_of`, `any_of`, `count`, `count_if`, `exclusive_scan`, `find`, `find_end`, `find_first_of`, `find_if`, `find_if_not`, `fill`, `for_each`, `inclusive_scan`, `mismatch`, `none_of`, `reduce`, `search`, `search_n`, `transform`, `transform_exclusive_scan`, `transform_inclusive_scan`, and `transform_reduce`. This was not meant to be an exhaustive test suite. Its purpose was to have enough coverage to get a sense for how much work it would be to implement this proposal.

Based on these test results, this proposal is implementable with reasonable effort. Some changes will have to be made to the compiler's standard library, but those changes should not be difficult to make. I expect that most of the effort will be spent writing tests, or adapting existing tests, to cover this area.

§ 10. Feature Test Macro

This change affects the well-formed-ness of certain code. Some users might want to write their code two different ways based on whether or not a standard library has implemented this change. Therefore, I believe that the benefit to users of a feature test macro is greater than the cost to implementers of defining it. This proposal adds the new macro `__cpp_lib_algorithm_iterator_requirements`.

§ 11. Wording

Changes are relative to N4888 from June 2021.

Change **25.2 "Algorithms requirements" [algorithms.requirements] paragraphs 4 and 5** as follows, combining them into a single paragraph:

Throughout this Clause, where the template parameters are not constrained, the names of template parameters are used to express type requirements.

- If an algorithm's *Effects*: element specifies that a value pointed to by any iterator passed as an argument is modified, then the type of that argument shall meet the requirements of a mutable iterator ([iterator.requirements]).
- If an algorithm's template parameter is named `InputIterator`, `InputIterator1`, or `InputIterator2`, the template argument shall meet the *Cpp17InputIterator* requirements ([input.iterators]).
- If an algorithm's template parameter is named `OutputIterator`, `OutputIterator1`, or `OutputIterator2`, the template argument shall meet the *Cpp17OutputIterator* requirements ([output.iterators]).
- If an algorithm's template parameter is named `ForwardIterator`, `ForwardIterator1`, ~~or~~ `ForwardIterator2`, or `NoThrowForwardIterator`, the template argument shall meet the *Cpp17ForwardIterator* requirements ([forward.iterators]) if it is required to be a mutable iterator, or model `forward_iterator` ([iterator.concept.forward]) otherwise .
- If an algorithm's template parameter is named `NoThrowForwardIterator`, the template argument ~~shall meet the *Cpp17ForwardIterator* requirements ([forward.iterators]), and~~ is also required to have the property that no exceptions are thrown from increment, assignment, or comparison of, or indirection through, valid iterators.
- If an algorithm's template parameter is named `BidirectionalIterator`, `BidirectionalIterator1`, or `BidirectionalIterator2`, the template argument shall meet the *Cpp17BidirectionalIterator* requirements ([bidirectional.iterators]) if it is required to be a mutable iterator, or model `bidirectional_iterator` ([iterator.concept.bidir]) otherwise .
- If an algorithm's template parameter is named `RandomAccessIterator`, `RandomAccessIterator1`, or `RandomAccessIterator2`, the template argument shall meet the *Cpp17RandomAccessIterator* requirements ([random.access.iterators]) if it is required to be a mutable iterator, or model `random_access_iterator` ([iterator.concept.random.access]) otherwise .

~~If an algorithm's *Effects*: element specifies that a value pointed to by any iterator passed as an argument is modified, then that algorithm has an additional type requirement: The type of that argument shall meet the requirements of a mutable iterator ([iterator.requirements]).~~

[*Note 1*: ~~This requirement does~~ These requirements do not affect iterator arguments that ~~are named `OutputIterator`, `OutputIterator1`, or `OutputIterator2`, because output iterators must~~

> ~~always be mutable, nor does it affect arguments that~~ are constrained, for which iterator category and mutability requirements are expressed explicitly. — *end note*]

> ISSUE 1 As currently worded, before these changes, the implementation is required to issue a diagnostic if the iterator type doesn't meet the given requirements. The requirements contain some semantic elements, which the implementation cannot reliably check at compile time, making it difficult to always issue the required diagnostic. The proposed wording change does not fix this issue, since the concepts also have semantic requirements that can't be reliably checked at compile time. Should the wording be changed to make the failure to meet the iterator requirements IFNDR or undefined behavior?

Change **25.7.12 "Sample" [alg.random.sample] paragraphs 2 and 5** as follows:

Paragraph 2:

> *Preconditions*: `out` is not in the range `[first, last)`. For the overload in namespace `std`:
>
> - `PopulationIterator` meets the *Cpp17InputIterator* requirements ([input.iterators]).
> - `SampleIterator` meets the *Cpp17OutputIterator* requirements ([output.iterators]).
> - `SampleIterator` meets the *Cpp17RandomAccessIterator* requirements ([random.access.iterators]) unless `PopulationIterator` ~~meets the *Cpp17ForwardIterator* requirements ([forward.iterators])~~ models `forward_iterator`([iterator.concept.forward]).
> - `remove_reference_t<UniformRandomBitGenerator>` meets the requirements of a uniform random bit generator type ([rand.req.urng]).

Paragraph 5:

> *Remarks*:
>
> - For the overload in namespace `std`, stable if and only if `PopulationIterator` ~~meets the *Cpp17ForwardIterator* requirements~~ models `forward_iterator` . For the first overload in namespace `ranges`, stable if and only if `I` models `forward_iterator`.
> - To the extent that the implementation of this function makes use of random numbers, the object `g` serves as the implementation's source of randomness.

Change **25.7.9 "Unique" [alg.unique] paragraph 8.2.2** as follows:

> - For the overloads with no `ExecutionPolicy`, let `T` be the value type of `InputIterator`. If `InputIterator` ~~meets the *Cpp17ForwardIterator* requirements~~ models `forward_iterator` ([iterator.concept.forward]) , then there are no additional requirements for `T`. Otherwise, if `OutputIterator` meets the *Cpp17ForwardIterator* requirements and its value type is the same as `T`, then `T` meets the *Cpp17CopyAssignable* (Table 31) requirements. Otherwise, `T` meets both the *Cpp17CopyConstructible* (Table 29) and *Cpp17CopyAssignable* requirements.

Change **30.10.2 "`regex_match`" [re.alg.match] paragraph 1** as follows:

> *Preconditions*: `BidirectionalIterator` ~~meets the *Cpp17BidirectionalIterator* requirements ([bidirectional.iterators])~~ models `bidirectional_iterator` ([iterator.concept.bidir]) .

Change **30.10.3 "`regex_search`" [re.alg.search] paragraph 1** as follows:

> *Preconditions*: `BidirectionalIterator` ~~meets the *Cpp17BidirectionalIterator* requirements ([bidirectional.iterators])~~ models `bidirectional_iterator` ([iterator.concept.bidir]) .

Add the following feature test macro to **[version.syn]**:

> `#define __cpp_lib_algorithm_iterator_requirements` *date // also in* `<algorithm>`, `<numeric>`, `<memory>`

§ References

§ Informative References

[P2214]
Barry Rezvin; Conor Hoekstra; Tim Song. *A Plan for C++23 Ranges*. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p2214r0.html

[P2321]
Tim Song. *zip*. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2321r2.html

§ Issues Index

ISSUE 1 As currently worded, before these changes, the implementation is required to issue a diagnostic if the iterator type doesn't meet the given requirements. The requirements contain some semantic elements, which the implementation cannot reliably check at compile time, making it difficult to always issue the required diagnostic. The proposed wording change does not fix this issue, since the concepts also have semantic requirements that can't be reliably checked at compile time. Should the wording be changed to make the failure to meet the iterator requirements IFNDR or undefined behavior? ↲